

THREAD INTERRUPT :

22 February 2025 14:09

What is Thread Interrupt?

- It is used to interrupt an executing thread.
- Interrupt method will only work when thread is in sleeping or waiting state.
- In Java, a thread can be interrupted to **politely** request it to stop its execution. The **interrupt()** method is used to signal a thread that it should stop what it is doing. However, **interrupting a thread does not forcibly stop it; it only sets a flag (interrupt status) that the thread can check**.
- If thread is not in sleeping or waiting state then calling this method will perform normal behaviour.
- When we use intrrupt() then It will throw an interrupted exception.

Why Do We Need Thread Interrupt?

- To stop a thread without using stop() (which is unsafe and deprecated).
- To gracefully terminate long-running threads (e.g., background tasks, heavy computations).
- To handle thread termination in a controlled way instead of forcing it to stop.

Real-Life Example (Online Shopping Order Cancellation)

Imagine you are placing an **online order**, but suddenly, you **cancel it before checkout**. The server was processing your order, but since you clicked "Cancel", the process stops.

CODE:

```
package multithreading;
class OrderProcessing extends Thread {
    public void run() {
        for (int i = 1; i <= 10; i++) {
            System.out.println("Processing order step " + i);

            try {
                Thread.sleep(1000); // Simulating order processing delay
            } catch (InterruptedException e) {
                System.out.println("Order processing was interrupted!");
                return;
            }
        }
        System.out.println("Order successfully placed!");
    }
}

public class OnlineShoppingExample {
    public static void main(String[] args) {
        OrderProcessing order = new OrderProcessing();
        order.start();

        try {
            Thread.sleep(5000); // Simulate user deciding to cancel order
            order.interrupt(); // User cancels the order
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- The thread **simulates order processing** step-by-step.
- If the user **interrupts (cancels)** within 5 seconds, the process **stops**.
- The system **gracefully handles** the interruption and cancels the order.

- Interrupted() and isInterrupted() :

Both method is used to check whether a thread is interrupted or not .

1. **interrupted()** -> clears the **interrupt status true to false if thread is interrupted**
2. **isInterrupted()** -> does not change the interrupted status.

1. interrupted() : will change the result.
2. isInterrupted() : will not change the result.

1 What is interrupt()?

- interrupt() is a method used to signal a thread **to stop its execution**.
- It **does not forcefully stop the thread** but sets an internal **interrupt flag** to indicate that the thread should stop.
- The thread must **explicitly check** the flag using isInterrupted() or handle InterruptedException inside catch.

2 What is `isInterrupted()`?

- `isInterrupted()` checks whether the thread **has been interrupted**.
- It **does not clear the interrupt flag**, so the flag remains true until manually reset.
- This allows a thread to decide **when and how** to stop.